Лабораторная работа 3
Технология MapReduce

Вариант 11: DevOps-инженер

Студент: Сидельникова Дарья Николаевна
Группа: ИИИ-3-ИнжДан-1-2
Преподаватель: Артём Галимьянов
Дата выполнения: 21 октября 2026 года

Репозиторий: https://github.com/DariaSide/lab3

Цель

Развернуть объектное хранилище RustFS, обработать один и тот же S3-объект собственной реализацией MapReduce и DuckDB, доказать совпадение результатов, измерить время и память, а затем получить практические выводы о навыках, зарплатах и регионах.

1. Исходные данные и условие варианта

Исходный CSV занимает 847 619 384 байта и содержит 229 388 строк.

Для варианта использовано точное условие role = 'DevOps-инженер'. Поиск слова DevOps в названии не использовался: он захватывает системных инженеров, разработчиков и администраторов, поэтому нарушает индивидуальный вариант.

Показатель	Значение
Строк после фильтра	5158
Со списком навыков	3534
С зарплатой в RUR	1037
Формат сегмента	NDJSON, одна вакансия на строку
Размер объекта	1 628 741 байт

Правила очистки

Навыки разделяются по точке с запятой, пробелы нормализуются, повторы внутри вакансии удаляются без учёта регистра. Сочетания сортируются и считаются один раз на вакансию. Зарплата учитывается только в RUR: для интервала берётся середина, для одной границы - имеющееся значение; net переводится в gross делением на 0,87. Вакансии без зарплаты участвуют в частотах, но не в медианах.

Команда подготовки

```
$ .venv/bin/python -m src.prepare --input /Users/side/Downloads/it_vacancies_2024.csv
selected_rows: 5158
rows_with_skills: 3534
rows_with_salary: 1037
```

2. RustFS и объект в hh-lake

RustFS запущен в Docker в режиме Single Node Multiple Disk с четырьмя томами /data1 ... /data4. На Docker Desktop тома видны как один виртуальный блочный девайс, поэтому только для лаборатории включён RUSTFS_UNSAFE_BYPASS_DISK_CHECK. В производственной среде такой флаг использовать нельзя.

Состояние контейнера

```
$ docker ps --filter name=lab3-rustfs
lab3-rustfs Up 23 minutes (healthy) 127.0.0.1:9000-9001->9000-9001/tcp
```

Объекты бакета

```
$ .venv/bin/python -m src.storage list
28640 s3://hh-lake/results/11-devops/analysis.json
31817212 s3://hh-lake/results/11-devops/duckdb.json
31814453 s3://hh-lake/results/11-devops/mapreduce.json
3601489 s3://hh-lake/results/11-devops/mapreduce_pairs.json
220316 s3://hh-lake/results/11-devops/mapreduce_skills.json
27663468 s3://hh-lake/results/11-devops/mapreduce_triples.json
1126 s3://hh-lake/results/11-devops/profiling.json
195 s3://hh-lake/results/11-devops/verification.json
1628741 s3://hh-lake/variants/11-devops/vacancies.ndjson
```

Размеры четырёх дисков после загрузки исходного объекта и результатов

```
$ docker exec lab3-rustfs du -sk /data1 /data2 /data3 /data4
47624 /data1
47624 /data2
47624 /data3
47624 /data4
```

Каждый том занимает одинаковые 47 664 KiB. Это ожидаемо для EC 2+2 на четырёх дисках: объект делится на два data-фрагмента и два parity-фрагмента. Код хранит 2 единицы полезных данных в 4 фрагментах, коэффициент избыточности равен 4/2 = 2. Разница между логическим размером объектов и du включает метаданные, малые объекты и размер блоков файловой системы.

Физическое ограничение

RustFS видит четыре отдельных тома и распределяет между ними данные и фрагменты восстановления. Но все тома физически находятся внутри одного Docker Desktop на одном компьютере. Поэтому эксперимент показывает, как система переживает недоступность одного тома, но не защищает от поломки самого компьютера или общего диска. Для настоящей отказоустойчивости нужны разные физические диски, а лучше несколько серверов.

3. Собственная реализация MapReduce

Драйвер получает размер объекта через HEAD, делит диапазон на чанки по 256 KiB и запускает четыре mapper-потока. Каждый mapper делает S3 Range GET. Границы выравниваются по переводу строки: запись принадлежит чанку, содержащему её первый байт. Mapper формирует локальные счётчики, reducer сливает их в детерминированном порядке.

Этап	Действие
Map	Чтение Range-чанка, разбор NDJSON, генерация C(n,1), C(n,2), C(n,3)
Shuffle	Ключ - отсортированный кортеж навыков
Reduce	Сумма частот и объединение выборок зарплат
Output	Три JSON-результата в s3://hh-lake/results/11-devops/

Контрольные числа

Размер стека	Уникальных ключей
1 навык	1 560
2 навыка	22 286
3 навыка	152 375



Рост объясняется числом сочетаний внутри вакансии: для n навыков mapper создаёт n одиночных ключей, n(n-1)/2 пар и n(n-1)(n-2)/6 троек. Поэтому число потенциальных троек растёт кубически.

4. DuckDB и независимая верификация

DuckDB подключается к тому же объекту s3://hh-lake/variants/11-devops/vacancies.ndjson через расширение httpfs. Массив skills раскрывается UNNEST ... WITH ORDINALITY. Пары и тройки формируются self join по возрастающим позициям, поэтому каждая комбинация учитывается один раз.

```
CREATE VIEW vacancies AS
SELECT * FROM read_ndjson_auto('s3://hh-lake/variants/11-devops/vacancies.ndjson');

CREATE VIEW expanded AS
SELECT vacancy_id, area, salary_rur, skill, pos
FROM vacancies, UNNEST(skills) WITH ORDINALITY AS t(skill, pos);
```

Полная проверка

Проверка	Результат
Число вакансий	5158 = 5158
Вакансии с навыками	3534 = 3534
Навыки: частоты, salary_n, медианы	Совпали
Пары: частоты, salary_n, медианы	Совпали
Тройки: частоты, salary_n, медианы	Совпали

Итог: verification.passed = true. Сравнивались не только итоговые суммы, а полные множества ключей и значений для всех трёх размеров стека.

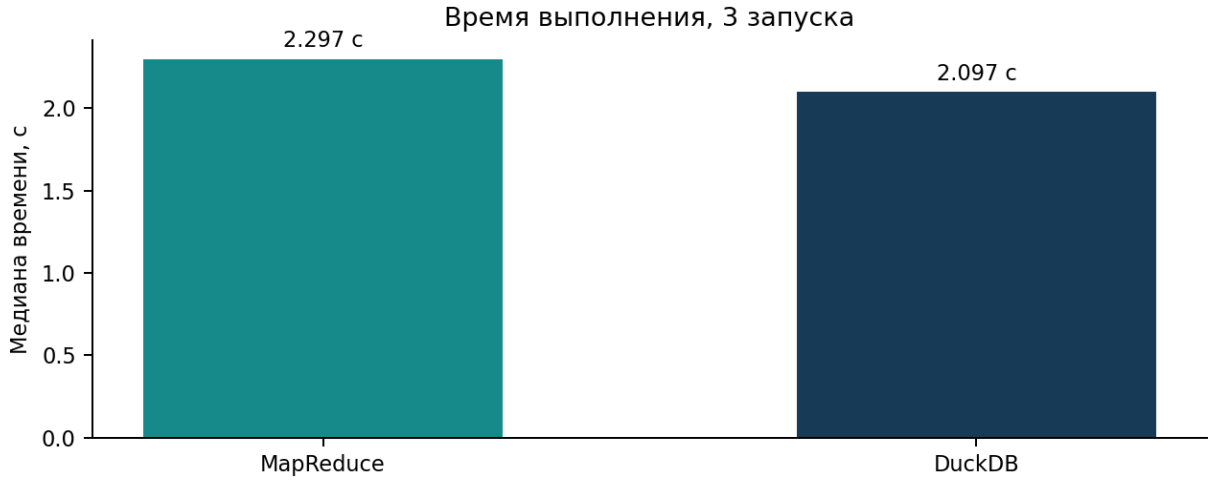
Найденная и исправленная ошибка

Первый запуск MapReduce дал 5 159 строк против 5 158 в DuckDB. Причина: строка на границе двух Range-чанков учитывалась дважды. После привязки записи к чанку по начальному байтовому смещению все таблицы совпали. Этот случай показывает практическую ценность независимого SQL-контроля.

5. Профилирование

Каждый способ запущен три раза. Измерено реальное wall-clock время и максимальный RSS всего дерева процесса с частотой опроса 20 мс.

Способ	Попытка	Время, с	Пик RSS, MiB
mapreduce	1	2.3046	556.70
mapreduce	2	2.2971	551.75
mapreduce	3	2.2746	595.52
duckdb	1	2.1382	677.12
duckdb	2	2.0968	688.00
duckdb	3	2.0674	708.34



Медиана MapReduce: 2.2971 с и 556.70 MiB. Медиана DuckDB: 2.0968 с и 688.00 MiB.

DuckDB быстрее примерно на 9%, но использует примерно на 24% больше памяти. На сегменте в 1,6 МБ накладные расходы планирования, HTTP и сериализации доминируют. Эти числа нельзя напрямую переносить на заявленные в задании 24 ГБ: там преимущества векторного SQL и ограничения памяти проявятся иначе.

6. Проверка отказа маппера

Для проверки повторного выполнения чанк 0 намеренно завершается исключением. Драйвер перезапускает только упавший mapper и продолжает reduce.

```
$ .venv/bin/python -m src.mapreduce_job --simulate-failure
{
"rows": 5158,
"rows_with_skills": 3534,
"unique_combinations": {
"1": 1560,
"2": 22286,
"3": 152375
},
"chunks": 7,
"workers": 4,
"retried_chunks": 1
}
```

После повторного запуска retried_chunks = 1, число строк осталось 5 158. Повторная команда src.verify снова вернула passed = true для навыков, пар и троек.

Семантика

Mapper не имеет побочных эффектов и выдаёт локальный аккумулятор, поэтому его повтор безопасен. Reducer запускается после завершения всех mapper-задач. Для промышленного решения потребовались бы лимит повторов, backoff, журнал причин отказа и промежуточные durable-результаты.

7. Региональный анализ

Выбраны три региона с наибольшим числом вакансий, чтобы уменьшить нестабильность медиан.

Регион	Вакансий	С зарплатой	Медиана gross, RUR	Топ-3 навыка
Москва	3007	523	287 356	Linux, Kubernetes, Docker
Санкт-Петербург	686	147	230 000	Linux, Docker, Kubernetes
Екатеринбург	188	41	172 414	Linux, Docker, Kubernetes

Наблюдения

Москва концентрирует 3 007 вакансий, или 58,3% сегмента, и имеет медиану 287 356 RUR по 523 объявлениям с зарплатой. В Санкт-Петербурге медиана равна 230 000 RUR, в Екатеринбурге - 172 414 RUR. Во всех трёх регионах лидируют Linux, Docker и Kubernetes, но порядок различается: в Москве Kubernetes опережает Docker, а в Санкт-Петербурге и Екатеринбурге Docker находится выше.

Самая частая пара во всех трёх регионах - Docker + Kubernetes. Самая частая тройка - Docker + Kubernetes + Linux. Это указывает на единое ядро DevOps-стека, а не на регионально разные специализации.

Ограничение

Зарплата указана только у части вакансий: 17,4% в Москве, 21,4% в Санкт-Петербурге и 21,8% в Екатеринбурге. Региональные медианы описывают объявления с раскрытой зарплатой и могут быть смещены относительно всего рынка.

8. Влияние навыков на зарплату

Для пары базой служит более частый из двух одиночных навыков. Для тройки - наиболее частая из трёх входящих пар. Сравниваются медианы только при наличии не менее 10 зарплат и у комбинации, и у базы. Это соответствует требованию не сравнивать редкую комбинацию с ещё более редкой базой.

Наибольшее повышение: пары

Комбинация	n зарплат	Медиана	База	Медиана базы	Разница
DevOps, SRE	10	400 000	DevOps	229 885	170 115
Git, GitLab CI	10	379 310	Git	218 391	160 920
Grafana, OpenShift	11	402 299	Grafana	264 368	137 931
Bash, СУБД	10	363 649	Bash	229 885	133 764
Docker, OpenShift	10	370 690	Docker	241 379	129 310

Наибольшее понижение: пары

Комбинация	n зарплат	Медиана	База	Медиана базы	Разница
Ansible, Atlassian Jira	10	143 678	Ansible	287 356	-143 678
Ansible, Debian	11	160 920	Ansible	287 356	-126 437
Debian, Nginx	12	114 943	Nginx	229 885	-114 943
Debian, Linux	19	114 943	Linux	229 885	-114 943
Debian, Администрирование серверов Linux	10	114 943	Администрирование серверов Linux	225 000	-110 057

Интерпретация

Связка DevOps + SRE показывает +170 115 RUR к медиане более частого DevOps, а Git + GitLab CI - +160 920 RUR к Git. Снижение у Ansible + Atlassian Jira и Ansible + Debian не означает причинного эффекта навыка: оно может отражать другой уровень позиций, отрасль, регион или особенности небольшой выборки.

9. Тройки навыков

Наибольшее повышение: тройки

Комбинация	n зарплат	Медиана	База	Медиана базы	Разница
CI/CD, ELK, Python	13	402 299	CI/CD, Python	229 885	172 414
Git, GitLab CI, Linux	10	379 310	Git, Linux	206 897	172 414
Bash, CI/CD, ELK	16	402 299	Bash, CI/CD	235 632	166 667
CI/CD, ELK, Gitlab	18	388 649	CI/CD, Gitlab	229 885	158 764
CI/CD, Python, СУБД	11	385 057	CI/CD, Python	229 885	155 172

Наибольшее понижение: тройки

Комбинация	n зарплат	Медиана	База	Медиана базы	Разница
ELK, Prometheus, Администрирование серверов Linux	19	143 678	ELK, Prometheus	287 356	-143 678
Kubernetes, PostgreSQL, Английский язык	10	146 839	Kubernetes, PostgreSQL	287 356	-140 517
Ansible, PostgreSQL, Английский язык	10	146 839	Ansible, PostgreSQL	287 356	-140 517
DevOps, MySQL, Terraform	12	195 402	DevOps, Terraform	330 460	-135 057
DevOps, MySQL, Prometheus	14	163 793	DevOps, Prometheus	287 356	-123 563

Вывод

CI/CD + ELK + Python и Git + GitLab CI + Linux имеют прибавку около 172 414 RUR к своим наиболее частым парным базам. Наблюдаемые различия полезны как гипотезы для поиска вакансий, но не доказывают, что добавление навыка само по себе повысит зарплату: данные наблюдательные, а зарплатные поля заполнены неполно.

10. Итоги и выбор инструмента

Вывод

На этом наборе данных мне удобнее использовать DuckDB. Запрос получается компактным, его проще проверить, а расчёт выполняется немного быстрее. Собственный MapReduce потребовал больше кода, зато на его примере хорошо видно весь процесс: чтение объекта частями, работа нескольких mapper-задач, объединение результатов и повтор упавшего чанка.

Если объём данных вырастет до десятков гигабайт, текущую реализацию MapReduce придётся доработать. Сейчас reducer хранит комбинации и списки зарплат в памяти. Для больших данных промежуточные результаты нужно частично записывать на диск и объединять поэтапно.

RustFS оказался удобным общим хранилищем для обоих способов расчёта: MapReduce и DuckDB читали один и тот же объект через S3 API. Четыре учебных тома показывают работу Erasure Coding, но для реальной системы я бы разместила данные на разных физических дисках или серверах, настроила резервное копирование, TLS и отдельную учётную запись для приложения.

Воспроизводимость

```
$ cp .env.example .env
$ make setup
$ make up
$ make prepare CSV=/absolute/path/to/it_vacancies_2024.csv
$ make upload
$ make profile
$ make analyze
$ .venv/bin/python -m src.storage upload-results
```